IP/Port/URL Allow-List & Egress Filtering Management for Microsoft 365 & Intune Endpoints — Government (GCC High)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 4

The problem . . . 4

The solution . . . 4

Why it matters . . . 4

What you get . . . 5

Bottom line . . . 5

Solution Overview — IP/Port/URL Allow-List & Egress Filtering Management for Microsoft 365 & Intune Endpoints (Government / GCC High) 5

What this solution does . . . 5

The one fact that governs this whole solution . . . 5

Commercial vs GCC High — what's different . . . 6

Objects this solution creates (name them, reuse them) . . . 7

The deployment model . . . 8

What's inside . . . 8

Bottom line . . . 9

Requirements Guide — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 9

R1. A confirmed GCC High tenant (not GCC / not Commercial) . . . 9

R2. US Government licensing (G3/G5 Gov or equivalent) . . . 9

R3. Administrative roles in the sovereign portals . . . 9

R4. Access to the GCC High endpoints web service . . . 10

R5. PowerShell 5.1+ and the Microsoft Graph module (USGov environment) . . . 10

R6. An egress device you can allow-list on (firewall / SWG / proxy / SASE) . . . 10

R7. Endpoints that must NOT be SSL-inspected (GCC High) . . . 10

R8. GCC High service/portal FQDNs to allow (the fixed facts) . . . 11

R9. A pilot group and rollback path (from Global Requirements) . . . 11

Implementation Guide — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

Phase 1 — Assess (derive today's required GCC High set) . . . 12

Phase 2 — Configure (build the named allow-list) . . . 12

Phase 3 — Pilot (apply to a small group only) . . . 14

Phase 4 — Validate (prove it works from a real endpoint) . . . 14

Phase 5 — Expand (fleet rollout + keep current) . . . 15

Keeping it current — the rule to remember . . . 15

Testing & Validation — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 15

Gate 1 — Assess . . . 16

Gate 2 — Configure . . . 16

Gate 3 — Pilot . . . 17
Gate 4 — Validate (from a pilot endpoint, behind the filtered egress) . . . 17
Gate 5 — Expand & keep current . . . 17
Regression test (run after any egress-device change) . . . 18
Troubleshooting — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 18
A. Wrong cloud / wrong instance . . . 18
B. Sign-in / authentication fails behind the proxy . . . 19
C. Intune enrollment / check-in stalls . . . 19
D. Defender for Endpoint / EPM devices never onboard . . . 19
E. Teams calls drop or have no audio/video . . . 19
F. Exchange / SharePoint access issues . . . 19
G. Web service returns HTTP 429 (Too Many Requests) . . . 19
H. Allow-list keeps going stale (periodic outages after month-start) . . . 20
I. Graph script fails to connect . . . 20
J. SSL-inspected endpoint breaks after enabling inspection . . . 20
Escalation checklist (before opening a ticket) . . . 20
Best Practices — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 20
Query the right cloud — every time . . . 21
Never hardcode Microsoft IP ranges . . . 21
Respect the SSL-inspection bypass list . . . 21
Pilot first, always . . . 21
Keep the 30-day window . . . 22
Name objects consistently and document them . . . 22
Least privilege and separation . . . 22
Watch for parity gaps . . . 22
Have a rollback ready . . . 22
Deliverables Summary — Egress Filtering for M365/Intune Endpoints (Government / GCC High) 23
Documents (`source/`) . . . 23
Scripts (`scripts/`) . . . 23
Named objects you will create . . . 24
Compiled deliverables (produced by the build, not by this authoring) . . . 24
Definition of done . . . 25
Appendix A — PowerShell Scripts 26

Executive Summary — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

The problem

Your users reach Microsoft 365, Intune, and Defender over a **filtered egress path** — a firewall, secure web gateway, or SASE/SSE stack. Those cloud services only work if the **exact IP ranges, URLs, and ports** they use are allowed through and, critically, if a specific set of endpoints is **not** SSL-inspected. In **GCC High**, this is a sovereign-cloud problem: the endpoints, portals, and service names are a **different published set** from the Commercial cloud. Configure it with Commercial values — the intuitive mistake — and you get **silent failures**: broken sign-in, stalled Intune check-ins, Defender devices that never onboard, and dropped Teams calls.

Two facts make this hard to get right and easy to get wrong:

1. **The GCC High endpoint set is separate.** It must be pulled from the `USGovGCCHigh` instance of Microsoft's IP/URL web service, and it points at `*.us` / `*.microsoft.us` endpoints — not `.com`.
1. **The data changes monthly.** Microsoft publishes additions **30 days in advance**. Any hardcoded list is a future outage waiting for the next update.

The solution

A repeatable, novice-friendly method to **derive** the required GCC High set from Microsoft's own web service, **build** a named allow-list and SSL-inspection bypass on your egress device from that derived data, **validate** connectivity from a real endpoint, and **keep it current** automatically inside the 30-day window. It ships with five PowerShell scripts (discovery, validation, reporting, safe remediation, and pilot scoping) that always query `USGovGCCHigh` and use the US Gov Graph environment, plus a full document set with a **Commercial-vs-GCC-High** parity map and cited Gov references.

Delivery follows a gated model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — so nothing goes fleet-wide until it is proven on a pilot with a documented rollback.

Why it matters

- **No silent failures.** Right instance, right sovereign portals and service FQDNs, right no-inspect list — the configuration matches the cloud you actually run.
- **No future outage.** Nothing is hardcoded; the allow-list is machine-maintained from the live Gov data, and additions are caught inside their 30-day advance window.
- **Audit-ready.** Named objects, dated reports, and a change record suit a government compliance context.
- **Low risk.** Read-only discovery and validation; change-capable scripts default to preview (`-WhatIf`); pilot-first rollout with a rollback.

What you get

Ten authored documents (overview, requirements, implementation, testing, troubleshooting, best practices, deliverables, this summary, cited parity notes, and the shared Network global requirements), five GCC High–targeted PowerShell scripts, and a clear set of named objects to build. Self-guided, tested, and a fraction of the cost of a consulting engagement.

Bottom line

Egress filtering for Microsoft 365, Intune, and Defender deployed **correctly on GCC High** — right endpoints, right portals, parity gaps handled, SSL-inspection bypasses in place — **proven on a pilot before you scale.**

Solution Overview — IP/Port/URL Allow-List & Egress Filtering Management for Microsoft 365 & Intune Endpoints (Government / GCC High)

What this solution does

Microsoft 365, Intune, and Microsoft Defender for Endpoint are **cloud services reached over the public network**. When your egress path is filtered — a firewall, a secure web gateway (SWG), a proxy, or a SASE/SSE stack — the client only works if the **exact IP ranges, URLs, and ports** that those services use are allowed through. Miss one, over-inspect one, or hardcode an address that later changes, and users see broken sign-in, stalled Intune check-ins, failed Teams calls, or Defender devices that never onboard.

This solution gives a novice administrator a **repeatable, click-to-implement method** to:

1. **Derive** the current required IP/URL/port set **from Microsoft's own web service** — not from a hand-typed list that goes stale.
1. **Build** a named, documented allow-list (and SSL-inspection **bypass** list) on your egress device from that derived set.
1. **Validate** connectivity from a real endpoint before and after each change, with pass/fail gates.
2. **Keep it current** automatically, because Microsoft publishes additions **30 days in advance** and changes the data monthly.

The critical difference in this edition: everything is derived from the **US Government GCC High instance** of the endpoints service and points at the **US sovereign (`*.us` / `*.microsoft.us`) admin portals and service endpoints** — not the Commercial (Worldwide) ones. GCC High is a **physically and logically separate cloud** with its **own published endpoint set**. Using Commercial endpoints on a GCC High tenant is a silent-failure trap: sign-in redirects, portal URLs, and service FQDNs are all different.

The one fact that governs this whole solution

Microsoft publishes four **instances** of the IP/URL web service, and **you must query the one that matches your tenant**:

Instance name	Cloud	Use it if…
`Worldwide`	Commercial (incl. GCC)	Standard commercial or GCC (moderate) tenant
`USGovGCCHigh`	**GCC High**	**This edition — US sovereign GCC High tenant**
`USGovDoD`	DoD	US Department of Defense tenant
`China`	21Vianet	Operated by 21Vianet in China

> *This solution uses `USGovGCCHigh` everywhere. The valid instance names are documented by Microsoft on the IP Address and URL web service page. (Cited — see `gcc-high-parity-notes.md`.)*

Because the addresses **change month to month**, this solution **never hardcodes Microsoft IPs**. Every script derives the live set from the `USGovGCCHigh` instance at run time. The only fixed facts we rely on are Microsoft's **published GCC High portal/service FQDNs** (e.g. `login.microsoftonline.us`, `*.manage.microsoft.us`, `security.microsoft.us`), which Microsoft documents explicitly for the Gov cloud.

Commercial vs GCC High — what's different

This is the section to read before you touch anything. If you have done this on a Commercial tenant, **almost every URL you memorized is wrong here.**

Area	Commercial (Worldwide)	**GCC High (this edition)**
Endpoints web-service instance	`Worldwide`	`USGovGCCHigh`
Endpoints download (JSON)	`endpoints.office.com/endpoints/Worldwide?...`	`endpoints.office.com/endpoints/USGOVGCCHigh?...`
Sign-in / identity	`login.microsoftonline.com`	`login.microsoftonline.us`
Microsoft 365 portal	`portal.office.com`	`portal.office365.us`
Azure portal	`portal.azure.com`	`portal.azure.us`
Intune admin center	`intune.microsoft.com` / `endpoint.microsoft.com`	`intune.microsoft.us`

Area	Commercial (Worldwide)	**GCC High (this edition)**
Intune service (client)	`*.manage.microsoft.com`	`*.manage.microsoft.us`
Intune Company Portal	`portal.manage.microsoft.com`	`portal.manage.microsoft.us`
Defender portal	`security.microsoft.com`	`security.microsoft.us`
Compliance / Purview	`compliance.microsoft.com`	`compliance.microsoft.us` / `purview.microsoft.us`
Exchange Online	`outlook.office.com`	`outlook.office365.us`
SharePoint / OneDrive	`*.sharepoint.com`	`*.sharepoint.us`
Teams	`*.teams.microsoft.com`	`*.gov.teams.microsoft.us`
Microsoft Graph	`graph.microsoft.com`	`graph.microsoft.us`
Defender/EPM device channel	`*.dm.microsoft.com`	`*.dm.microsoft.us`
Graph PowerShell environment	(default)	`Connect-MgGraph -Environment USGov`

> ***Parity gaps are real.*** *Some Commercial capabilities lag, are limited, or are absent in GCC High. Where a step depends on a capability whose GCC High behavior we could not confirm against Microsoft Gov documentation, it is flagged* ***"verify for GCC High"*** *at the step. Never assume a Commercial feature exists in GCC High just because the portal looks similar.*

Objects this solution creates (name them, reuse them)

Consistent names make the allow-list auditable and let every later step reference the same objects.

Object	Suggested name	What it is
Derived endpoint export	`EGRESSG-Endpoints-USGovGCCHigh.json`	The raw GCC High set pulled from the web service
Allow-list address group	`EGRESSG-M365-GCCHigh-Allow`	Firewall/SWG object holding the required IP ranges

Object	Suggested name	What it is
URL allow / bypass category	`EGRESSG-M365-GCCHigh-URLs`	Proxy/SWG URL category for the required FQDNs
SSL-inspection bypass list	`EGRESSG-NoInspect-GCCHigh`	FQDNs Microsoft says must **not** be SSL-inspected
Pilot device group (Entra)	`EGRESSG-Pilot`	Staged rollout group (see Global Requirement G6)
Change/version checkpoint file	`EGRESSG-latestversion.txt`	Stores the last-applied web-service version

The deployment model

This solution follows the standard model used across the library, with a **gate** between each phase — you do not advance until the gate passes.

Assess → **Configure** → **Pilot** → **Validate** → **Expand**

1. **Assess** — Confirm you are on a GCC High tenant, gather the Gov portal URLs, and run read-only discovery to derive today's required `USGovGCCHigh` set. *Gate: discovery returns a non-empty set from the `USGovGCCHigh` instance.*

1. **Configure** — Build the named allow-list, URL category, and SSL bypass list from the derived set. *Gate: every required endpoint set is represented; nothing hardcoded.*

1. **Pilot** — Apply to the `EGRESSG-Pilot` group only. *Gate: pilot devices pass validation.*

2. **Validate** — Run the validation script from a pilot endpoint; confirm sign-in, Intune check-in, Defender onboarding, and Teams media. *Gate: all checks green.*

1. **Expand** — Roll out fleet-wide, then schedule the monthly keep-current job. *Gate: scheduled job runs and reports "up to date" or a reviewed change set.*

What's inside

• **Requirements Guide** (`02-requirements.md`) — every prerequisite with **Verify** and **Implement-if-missing** steps, GCC High–specific.

• **Implementation Guide** (`03-implementation.md`) — derive the Gov required set, build the allow-list, validate, and keep current, step by step.

• **Testing** (`05-testing.md`) — pass/fail gates for each phase.

• **Troubleshooting** (`06-troubleshooting.md`) — GCC High–specific gotchas.

• **Best Practices** (`07-best-practices.md`), **Deliverables Summary** (`08-deliverables-summary.md`),

Executive Summary, and a cited **GCC High Parity Notes** reference.

- **5 PowerShell scripts** — Discovery, Validation, Report, Remediation (`-WhatIf` by default), and Set-ScopeTargeting — all querying the `USGovGCCHigh` instance and using `-Environment USGov` for Graph.

Bottom line

IP/Port/URL allow-list and egress filtering deployed **correctly on GCC High** — right web-service instance, right sovereign portals and service FQDNs, SSL-inspection bypasses in place, parity gaps called out, and a monthly keep-current job — **proven on a pilot before you scale.**

Requirements Guide — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

These are the prerequisites specific to **this** solution. They sit **on top of** the category-level **Global Requirements — Network** (`global-requirements-network.md`); confirm those first. Each entry below gives **Description · Why · Verify · Implement-if-missing**, written for the **GCC High** cloud.

> ***Sovereign-cloud reminder:*** *every portal and service URL in this edition is the* `*.us` */* `*.microsoft.us` *sovereign endpoint. If a Verify step sends you to a* `.com` *portal, you are in the wrong cloud. Where GCC High parity could not be confirmed, the step is flagged* ***"verify for GCC High."***

R1. A confirmed GCC High tenant (not GCC / not Commercial)

Description: Your Microsoft 365 tenant is a **US Government GCC High** tenant. **Why:** GCC High has its **own published endpoint set** (`USGovGCCHigh` instance) and its own portal URLs. GCC (moderate) uses the **Worldwide** instance; Commercial uses **Worldwide**. Getting this wrong means you allow-list the wrong cloud. **Verify:** Sign in at `https://portal.office365.us` and `https://portal.azure.us`. In the Microsoft Intune admin center `https://intune.microsoft.us`, confirm the portal loads under the `.us` domain. Your sign-in should redirect through `login.microsoftonline.us`. **Implement:** If you sign in at `.com` portals, you are **not** on GCC High — stop and confirm your tenant type with your licensing/onboarding contact before proceeding. Do not apply this solution to a Commercial tenant.

R2. US Government licensing (G3/G5 Gov or equivalent)

Description: Microsoft 365 US Government plans (e.g. **G3/G5 Government**) with Intune and, for the Defender steps, Microsoft Defender for Endpoint for US Government. **Why:** The service areas you allow-list (Exchange, SharePoint, Teams, Common, Intune, Defender) must actually be licensed and provisioned in your tenant. **Verify:** `https://portal.office365.us` → **Billing** → **Licenses**; confirm Gov SKUs are assigned. Confirm Intune is provisioned at `https://intune.microsoft.us`. **Implement:** Assign the appropriate US Government license plans to users/devices in scope.

R3. Administrative roles in the sovereign portals

Description: Least-privilege admin roles to read tenant configuration and (for remediation) configure the egress device. **Why:** Discovery and validation are read-only, but building the allow-list requires rights on your firewall/SWG, and scoping requires Entra group management. **Verify:** Confirm you can open

`https://intune.microsoft.us`, `https://security.microsoft.us`, and `https://entra.microsoft.com` (Entra portal is shared) with at least reader access, and that you have admin rights on the egress device. **Implement:** Assign least-privilege roles (e.g. **Global Reader** for discovery; the specific network-device role for allow-list changes). Reserve Global Administrator for tenant-wide changes only.

R4. Access to the GCC High endpoints web service

Description: The machine that runs the scripts must be able to reach `https://endpoints.office.com` and query the `USGOVGCCHigh` instance. **Why:** The entire required set is **derived** from this web service. No hardcoded Microsoft IPs are used anywhere in this solution. **Verify:** From the script host, in a browser or PowerShell, open `https://endpoints.office.com/version/USGOVGCCHigh?clientrequestid=<generate-a-guid>` and confirm a JSON version record for instance `USGOVGCCHigh` is returned. (Generate your own GUID — see `03-implementation.md`; do not reuse example GUIDs.) **Implement:** Allow `endpoints.office.com` outbound (TCP 443) from the script host. This host is typically a management/jump box, **not** a filtered user endpoint.

R5. PowerShell 5.1+ and the Microsoft Graph module (USGov environment)

Description: Windows PowerShell 5.1 or PowerShell 7+, plus the **Microsoft.Graph** module for the scope-targeting script. **Why:** Scripts call the web service via `Invoke-RestMethod` and (for Set-ScopeTargeting) manage the pilot group via Graph using `-Environment USGov`. **Verify:** `$PSVersionTable.PSVersion` shows 5.1+; `Get-Module Microsoft.Graph -ListAvailable` returns the module; `Connect-MgGraph -Environment USGov` connects to the Gov endpoint. **Implement:** `Install-Module Microsoft.Graph -Scope CurrentUser`. Always pass `-Environment USGov` — the default targets Commercial Graph and will fail on a GCC High tenant.

R6. An egress device you can allow-list on (firewall / SWG / proxy / SASE)

Description: The filtering device or service that controls outbound traffic (Netskope SWG per the Network category, or a firewall/proxy) with the ability to create **address groups**, **URL categories**, and **SSL-inspection bypass** rules. **Why:** The derived set is only useful if you can express it as named objects on the device. **Verify:** Confirm you can create an address object/group and a URL category, and that you can add FQDNs to an SSL-inspection **bypass** list. **Implement:** If your device cannot bypass SSL inspection per-FQDN, treat that as a blocker for the **no-inspect** endpoints below (Microsoft does **not** support inspecting them).

R7. Endpoints that must NOT be SSL-inspected (GCC High)

Description: Microsoft explicitly states SSL/TLS inspection is **not supported** on certain Gov endpoints; these must be on an inspection **bypass** list. **Why:** Inspecting them breaks Intune check-in, Defender/EPM device management, and enrollment. **Verify:** Confirm your bypass list includes at least the following Microsoft-documented Gov no-inspect endpoints:

- `*.manage.microsoft.us` (Intune) — SSL inspection **not supported**.
- `has.spserv.microsoft.com` — SSL inspection **not supported**.

- `*.dm.microsoft.us` (Defender for Endpoint **and** Endpoint Privilege Management device channel) —

SSL inspection **not supported**.

Implement: Add these to `EGRESSG-NoInspect-GCCHigh` before enabling inspection anywhere near M365 traffic. (Cited — see `gcc-high-parity-notes.md`.)

R8. GCC High service/portal FQDNs to allow (the fixed facts)

Description: The published Gov portal and partner-service FQDNs that identity, Intune, and Defender depend on. Unlike Microsoft's rotating IP ranges, these FQDNs are **stable and documented**, so they are safe to reference by name. **Why:** Even a perfect IP allow-list fails if these named endpoints are blocked or misrouted. **Verify:** Confirm the following resolve and are allowed outbound (TCP 443, and 80 where noted):

- **Identity:** `login.microsoftonline.us`, `*.auth.microsoft.us`, `*.gov.us.microsoftonline.com`
- **Intune service/portals:** `*.manage.microsoft.us`, `portal.manage.microsoft.us`, `intune.microsoft.us`, `enterpriseregistration.microsoftonline.us`
- **Graph:** `graph.microsoft.us`, `graph.microsoftazure.us`, `directory.microsoftazure.us`
- **Defender / security / compliance:** `*.security.microsoft.us`, `security.microsoft.us`, `compliance.microsoft.us`, `purview.microsoft.us`, `*.dm.microsoft.us`
- **Core workloads:** `outlook.office365.us`, `*.sharepoint.us`, `*.gov.teams.microsoft.us`, `*.gov.online.office365.us`, `www.office365.us`
- **Unified domains:** `*.usgovcloud.microsoft`, `*.usgovcloud-static.microsoft`, `*.usgovcloud-usercontent.microsoft`

Implement: Add any missing FQDNs to `EGRESSG-M365-GCCHigh-URLs`. Ports and the authoritative, current list are **derived** by the Discovery script from the `USGovGCCHigh` instance — this list is the human-readable cross-check, not a substitute for the derived data. (Cited — see `gcc-high-parity-notes.md`.)

R9. A pilot group and rollback path (from Global Requirements)

Description: An Entra security group `EGRESSG-Pilot` and a documented rollback (revert the allow-list / disable inspection changes). **Why:** Egress changes affect live traffic; every rollout is piloted and reversible (Global Requirements **G6**, **G7**). **Verify:** `EGRESSG-Pilot` exists in Entra Groups; a written rollback is approved. **Implement:** Create the group (`Set-ScopeTargeting` can create/populate it) and write the rollback before enabling enforcement.

Requirement gate: Do not proceed to Implementation until R1 (confirmed GCC High), R4 (web-service reachable, `USGOVGCCHigh` returns a version), and R7 (no-inspect list understood) all pass.

Implementation Guide — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

This guide takes you from nothing to a validated, self-maintaining allow-list for a **GCC High** tenant. It follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a **gate** at the end of each phase. Every step derives Microsoft's required set from the `USGovGCCHigh` instance — **no Microsoft IPs are ever typed by hand.**

> ***Before you start:*** *confirm the requirement gate in* `02-requirements.md` *passed (R1 GCC High confirmed, R4 web service reachable, R7 no-inspect understood). Keep the object names from* `01-solution-overview.md` *— you will reference them repeatedly.*

Phase 1 — Assess (derive today's required GCC High set)

Step 1.1 — Generate a stable ClientRequestId GUID

The web service requires a **ClientRequestId** GUID per calling machine. Generate one and reuse it (the scripts store it in `EGRESSG-latestversion.txt`).

```
[GUID]::NewGuid().Guid
```

> *Do **not** reuse the GUIDs shown in Microsoft's examples — Microsoft warns they may be blocked. Use your own.*

Step 1.2 — Confirm you are querying the GCC High instance

In a browser or PowerShell on the script host, check the version of the `USGOVGCCHigh` instance (replace `your-guid` with the GUID from Step 1.1):

```
https://endpoints.office.com/version/USGOVGCCHigh?clientrequestid=your-guid
```

You should get a JSON record whose `instance` is `USGOVGCCHigh`. If you see `Worldwide`, you queried the wrong instance — fix the URL. **This is the single most important check in the whole solution.**

Step 1.3 — Run Discovery (read-only)

Run the discovery script. It queries the `USGOVGCCHigh` endpoints web method, splits the result into IPv4/IPv6 ranges, URLs, and ports, flags the required and no-inspect endpoints, and writes `EGRESSG-Endpoints-USGovGCCHigh.json` plus a human-readable summary.

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Discovery.ps1
```

Read the summary. Confirm it lists sovereign FQDNs (`outlook.office365.us`, `*.manage.microsoft.us`, `*.sharepoint.us`, `login.microsoftonline.us`, etc.). If it shows `.com` M365 endpoints, you are on the wrong instance — stop.

> ***Gate 1 (Assess):*** *Discovery returns a **non-empty** set from the* `USGovGCCHigh` *instance and the summary shows* `*.us` *sovereign endpoints. Only then continue.*

Phase 2 — Configure (build the named allow-list)

You now translate the derived set into named objects on your egress device. **Do this on the device; the scripts derive and document the data, they do not reach into your firewall.** Use the object names from the overview so everything is auditable.

Step 2.1 — Create the IP address group `EGRESSG-M365-GCCHigh-Allow`

From `EGRESSG-Endpoints-USGovGCCHigh.json`, take the **IPv4/IPv6 ranges** for the `Optimize` and `Allow` category endpoints (these are the ones Microsoft marks required) and add them to a single named address group on your firewall/SWG:

- Object name: `EGRESSG-M365-GCCHigh-Allow`
- Contents: the derived CIDR ranges (paste from the Discovery export — never retype)
- Because these ranges **change monthly**, treat this object as **machine-maintained** (see Phase 5).

> *If you use ExpressRoute for Microsoft 365, note which endpoint sets are ER-supported (the export carries the* `expressRoute` *flag) and route those accordingly.* ***Verify for GCC High*** *which of your workloads are actually on ER in your environment.*

Step 2.2 — Create the URL category `EGRESSG-M365-GCCHigh-URLs`

Add the derived **URLs/FQDNs** (again from the export) to a proxy/SWG URL category or bypass list:

- Object name: `EGRESSG-M365-GCCHigh-URLs`
- Contents: the derived FQDNs, including the GCC High sovereign names and the **Unified domains** (`*.usgovcloud.microsoft`, `*.usgovcloud-static.microsoft`, `*.usgovcloud-usercontent.microsoft`).
- Cross-check against the fixed-fact FQDN list in `02-requirements.md` (R8). If the derived data is missing anything on that list, allow the named FQDN anyway and note it for review.

Step 2.3 — Create the SSL-inspection bypass list `EGRESSG-NoInspect-GCCHigh`

Add the endpoints Microsoft says must **not** be inspected (R7) to an SSL-inspection **bypass** list:

- Object name: `EGRESSG-NoInspect-GCCHigh`
- Contents: `*.manage.microsoft.us`, `has.spserv.microsoft.com`, `*.dm.microsoft.us`
- Ensure your inspection policy **excludes** this list **before** you enable inspection near M365 traffic. Inspecting these breaks Intune, Defender, and EPM.

Step 2.4 — Confirm required ports

From the export, confirm the ports are allowed for the corresponding objects. Typical GCC High: **TCP 443** (and **80** for many endpoints), **TCP 25/587/993/995/143** for Exchange mail protocols, and **UDP 3478–3481** for Teams media. Use the **derived** `tcpPorts`/`udpPorts` values — do not assume.

> ***Gate 2 (Configure):*** *Every required endpoint set from the export is represented across* `EGRESSG-M365-GCCHigh-Allow` *and* `EGRESSG-M365-GCCHigh-URLs`*; the no-inspect list is applied;* ***no Microsoft IP was typed by hand.*** *Run* `EGRESSG-Validation.ps1` *in report mode from the script host as a sanity check (it does not require pilot devices yet).*

Phase 3 — Pilot (apply to a small group only)

Step 3.1 — Build/confirm the pilot group

Create or confirm `EGRESSG-Pilot` and add a handful of representative devices/users.

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Set-ScopeTargeting.ps1 -GroupName "EGRESSG-Pilo
```

`Set-ScopeTargeting` connects to Graph with `-Environment USGov`. It defaults to `-WhatIf` — review the planned changes, then re-run with `-Execute` to apply.

Step 3.2 — Apply the egress policy to the pilot only

On your egress device, scope the new allow-list / inspection-bypass policy to the pilot devices' source (subnet, tag, or group) — **not** the whole fleet. This is the reversible blast-radius control required by Global Requirement **G7**.

> ***Gate 3 (Pilot):*** *Policy is live for* `EGRESSG-Pilot` *devices only, and a rollback is written and approved.*

Phase 4 — Validate (prove it works from a real endpoint)

Step 4.1 — Run Validation from a pilot endpoint

On a device inside the pilot (behind the filtered egress path), run:

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Validation.ps1
```

The validation script derives the current `USGovGCCHigh` set and tests reachability to representative Gov endpoints across each service area:

- **Identity:** `login.microsoftonline.us`
- **Intune:** an `*.manage.microsoft.us` host and `enterpriseregistration.microsoftonline.us`
- **Defender/EPM:** an `*.dm.microsoft.us` host
- **Exchange:** `outlook.office365.us`
- **SharePoint:** a `*.sharepoint.us` host
- **Teams:** a `*.gov.teams.microsoft.us` host (and UDP media ports where testable)
- **Graph:** `graph.microsoft.us`

Each check reports **PASS/FAIL**. It is read-only.

Step 4.2 — Confirm real service behavior

On the pilot device, confirm end-to-end: sign in via `login.microsoftonline.us`, the Intune Company Portal (`portal.manage.microsoft.us`) checks in, the device shows in Defender (`security.microsoft.us`), and a Teams call connects with working audio/video.

> ***Gate 4 (Validate):*** *All Validation checks are* ***PASS*** *and real sign-in / Intune check-in / Defender onboarding / Teams media all work on the pilot. Investigate any FAIL with* `06-troubleshooting.md` *before expanding.*

Phase 5 — Expand (fleet rollout + keep current)

Step 5.1 — Expand scope

Widen the egress policy from `EGRESSG-Pilot` to the full fleet in stages (e.g. by site). Re-run `EGRESSG-Validation.ps1` from a device in each new wave.

Step 5.2 — Schedule the monthly keep-current job

Microsoft updates the data monthly and publishes additions **30 days in advance**. Schedule Discovery to run (e.g. weekly) so you always have 30 days of lead time:

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Discovery.ps1
```

Discovery compares the live `USGovGCCHigh` version against the stored version in `EGRESSG-latestversion.txt`. If unchanged, it reports "up to date." If changed, it writes the new export and a **diff** of what to add/remove, so you can update `EGRESSG-M365-GCCHigh-Allow` / `EGRESSG-M365-GCCHigh-URLs` **before** the new endpoints go live.

Step 5.3 — Produce the audit report

Generate the point-in-time report for change control and audit:

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Report.ps1
```

Step 5.4 — Remediation (optional, safe by default)

If you maintain the device object exports as files, `EGRESSG-Remediation.ps1` can regenerate the derived allow-list export and flag drift. It defaults to `-WhatIf` and makes **no** change unless you pass `-Execute`. It never edits your firewall directly — it produces the corrected export for you to import.

```
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Remediation.ps1           # preview (WhatIf)
powershell -ExecutionPolicy Bypass -File .\scripts\EGRESSG-Remediation.ps1 -Execute   # write corrected ex
```

> ***Gate 5 (Expand):*** *Fleet is covered, validation passes per wave, and the keep-current job runs on a schedule and reports either "up to date" or a reviewed change set.*

Keeping it current — the rule to remember

- **Never hardcode Microsoft IPs.** They change monthly; a hardcoded list is a future outage.
- **Always query** `USGOVGCCHigh`**.** The default/Worldwide instance is the wrong cloud.
- **Watch the 30-day window.** Additions are published 30 days early — apply them within that window.
- **Keep the no-inspect list current.** If Microsoft adds a Gov no-inspect endpoint, add it to

`EGRESSG-NoInspect-GCCHigh` and **verify for GCC High** against current Gov docs.

Testing & Validation — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

Testing mirrors the deployment model. Each phase has an explicit **pass/fail gate**; do not advance until the gate is green. All tests here are **read-only** — they observe, they do not change the egress device. Every test derives its expectations from the `USGovGCCHigh` instance.

> ***Instance sanity check (run this first, always).*** *Confirm every derived expectation comes from the Gov cloud. Query* `https://endpoints.office.com/version/USGOVGCCHigh?clientrequestid=your-guid` *and confirm the returned* `instance` *is* `USGOVGCCHigh`*. If it is* `Worldwide`*, every downstream test is invalid.*

Gate 1 — Assess

Test	How	Pass
Correct cloud	Sign in at `portal.office365.us` / `intune.microsoft.us`	Portals load under `.us`; sign-in via `login.microsoftonline.us`
Web service reachable	`EGRESSG-Discovery.ps1` from the script host	Exits 0; writes `EGRESSG-Endpoints-USGovGCCHigh.json`
Right instance	Read the Discovery summary	`instance = USGovGCCHigh`; FQDNs are `*.us` sovereign names
Non-empty set	Count endpoint sets in the export	Greater than zero required endpoint sets

Fail action: if the summary shows `.com` M365 endpoints, you queried `Worldwide` — fix the instance and re-run.

Gate 2 — Configure

Test	How	Pass
Coverage	Compare device objects to the export	Every required endpoint set is in `EGRESSG-M365-GCCHigh-Allow` or `EGRESSG-M365-GCCHigh-URLs`
No hardcoding	Review how ranges were added	Ranges were **pasted from the export**, not retyped
No-inspect applied	Review inspection policy	`*.manage.microsoft.us`, `has.spserv.microsoft.com`, `*.dm.microsoft.us` are on `EGRESSG-NoInspect-GCCHigh` and excluded from inspection
Ports	Compare allowed ports to export	TCP 443/80, Exchange mail ports, Teams UDP 3478–3481 as the export dictates
Report mode	`EGRESSG-Validation.ps1` from the script host	Reachable endpoints report PASS

Gate 3 — Pilot

Test	How	Pass
Scope	Check the egress policy target	Policy applies to `EGRESSG-Pilot` source only, not the fleet
Group membership	`EGRESSG-Set-ScopeTargeting.ps1 -WhatIf`	Pilot group exists and lists the expected members
Rollback ready	Review change record	Written, approved rollback exists (Global Requirement G7)

Gate 4 — Validate (from a pilot endpoint, behind the filtered egress)

Run `EGRESSG-Validation.ps1` on a pilot device. Every row must be **PASS**.

Service area	Endpoint tested	Pass
Identity	`login.microsoftonline.us`	Reachable on 443
Intune service	`*.manage.microsoft.us` host	Reachable; **not** SSL-inspected
Enrollment	`enterpriseregistration.microsoftonline.us`	Reachable on 443
Defender / EPM	`*.dm.microsoft.us` host	Reachable; **not** SSL-inspected
Exchange	`outlook.office365.us`	Reachable on 443
SharePoint	`*.sharepoint.us` host	Reachable on 443
Teams signaling	`*.gov.teams.microsoft.us` host	Reachable on 443
Teams media	UDP 3478–3481 (where testable)	Reachable
Graph	`graph.microsoft.us`	Reachable on 443

Real-behavior confirmation (must also pass):

- Sign-in completes via `login.microsoftonline.us`.
- Intune Company Portal (`portal.manage.microsoft.us`) checks in successfully.
- Device appears healthy in Defender (`security.microsoft.us`).
- A Teams call connects with two-way audio/video.

Fail action: if an SSL-inspected endpoint fails, confirm it is on `EGRESSG-NoInspect-GCCHigh`. If identity fails, confirm `login.microsoftonline.us` (not `.com`) is allowed. See `06-troubleshooting.md`.

Gate 5 — Expand & keep current

Test	How	Pass
Per-wave validation	`EGRESSG-Validation.ps1` on a device in each wave	All PASS
Scheduled job runs	Check the scheduled Discovery task	Runs on schedule; writes report
Change detection	Compare stored vs live version	Reports "up to date" **or** a diff to review
30-day lead	Review any diff's effective dates	Additions applied within the 30-day advance window
Audit report	`EGRESSG-Report.ps1`	Produces a dated point-in-time report

Regression test (run after any egress-device change)

1. Instance sanity check (`USGOVGCCHigh`).
2. `EGRESSG-Discovery.ps1` — confirm "up to date" or apply the diff.
3. `EGRESSG-Validation.ps1` from a real endpoint — all PASS.
4. Spot-check one SSL-inspected-sensitive flow (Intune check-in) still works.

If all four pass, the change is safe. If any fail, roll back per G7 and diagnose.

Troubleshooting — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

GCC High–specific symptoms, root causes, and fixes. The most common root cause by far is **querying or allow-listing the wrong cloud** (`Worldwide`/`.com` instead of `USGovGCCHigh`/`.us`). Work top-down; each entry gives **Symptom** → **Likely cause** → **Fix**.

A. Wrong cloud / wrong instance

Symptom: Discovery output shows `.com` M365 endpoints (`outlook.office.com`, `*.manage.microsoft.com`), or sign-in redirects to `login.microsoftonline.com`. **Likely cause:** You queried the `Worldwide` instance, or the egress device allows Commercial FQDNs instead of Gov. **Fix:** Re-run against `USGOVGCCHigh`: `https://endpoints.office.com/version/USGOVGCCHigh?clientrequestid=your-guid` — confirm the returned `instance` is `USGOVGCCHigh`. Rebuild `EGRESSG-M365-GCCHigh-Allow` / `EGRESSG-M365-GCCHigh-URLs` from the corrected export. In GCC High, identity is `login.microsoftonline.us` — allow the `.us` name.

Symptom: Portals load at `.com` and everything "looks normal," but Gov services misbehave. **Likely cause:** You may not actually be on a GCC High tenant. **Fix:** Confirm tenant type (R1): sign in at `portal.office365.us` / `intune.microsoft.us`. If you can only sign in at `.com`, you are not on GCC High — do not apply this solution.

B. Sign-in / authentication fails behind the proxy

Symptom: Users cannot authenticate; token or portal errors. **Likely cause:** `login.microsoftonline.us`, `*.auth.microsoft.us`, or `*.gov.us.microsoftonline.com` is blocked, or the Commercial `login.microsoftonline.com` was allowed instead. **Fix:** Allow the Gov identity FQDNs (R8) on TCP 443. Confirm with `EGRESSG-Validation.ps1` (Identity row must PASS).

C. Intune enrollment / check-in stalls

Symptom: Devices don't enroll, Company Portal won't check in, or software updates fail to download. **Likely cause 1:** `*.manage.microsoft.us` is **being SSL-inspected** (not supported). **Fix:** Add `*.manage.microsoft.us` and `has.spserv.microsoft.com` to `EGRESSG-NoInspect-GCCHigh` and exclude them from inspection. **Likely cause 2:** Proxy requires **authentication** to `manage.microsoft.us`. **Fix:** Some Intune tasks require **unauthenticated** proxy access to `manage.microsoft.us` — allow it unauthenticated. **Likely cause 3:** `enterpriseregistration.microsoftonline.us` blocked. **Fix:** Allow it on TCP 443.

D. Defender for Endpoint / EPM devices never onboard

Symptom: Devices don't appear in `security.microsoft.us`, or EPM elevation rules never apply. **Likely cause:** `*.dm.microsoft.us` is blocked or **SSL-inspected** (inspection not supported). **Fix:** Allow `*.dm.microsoft.us` and add it to `EGRESSG-NoInspect-GCCHigh`. Confirm the Defender/EPM row in `EGRESSG-Validation.ps1` is PASS. If onboarding still fails, **verify for GCC High** whether your OS versions require the standard vs streamlined Gov connectivity URL set.

E. Teams calls drop or have no audio/video

Symptom: Teams connects to chat but calls fail or media is one-way. **Likely cause:** Teams media UDP ports (**3478–3481**) or `*.gov.teams.microsoft.us` are blocked. **Fix:** Allow UDP 3478–3481 to the derived Teams IP ranges and `*.gov.teams.microsoft.us` on 443. Remember GCC High Teams media routes only through **USGov Arizona / USGov Texas** datacenters.

F. Exchange / SharePoint access issues

Symptom: Outlook or SharePoint/OneDrive fails behind the proxy. **Likely cause:** Gov FQDNs blocked — Exchange is `outlook.office365.us` (not `outlook.office.com`); SharePoint is `*.sharepoint.us` (not `*.sharepoint.com`). **Fix:** Allow the Gov FQDNs and their derived IP ranges/ports (Exchange mail ports 25/143/587/993/995 where used).

G. Web service returns HTTP 429 (Too Many Requests)

Symptom: Discovery/Validation fails with 429 from `endpoints.office.com`. **Likely cause:** The endpoints/changes web methods are rate-limited per client IP. **Fix:** Wait 1 hour, or generate a new ClientRequestId GUID. As designed, the scripts only call the **endpoints** method when the **version** method shows a new version — don't disable that check, and don't run more than once an hour.

H. Allow-list keeps going stale (periodic outages after month-start)

Symptom: Connectivity breaks around the start of a month. **Likely cause:** Microsoft updated the GCC High set and your allow-list wasn't refreshed within the 30-day advance window; or someone hardcoded IPs. **Fix:** Schedule `EGRESSG-Discovery.ps1` (weekly), review the emitted diff, and apply additions before their effective date. Convert any hardcoded ranges to machine-maintained objects fed from the export.

I. Graph script fails to connect

Symptom: `EGRESSG-Set-ScopeTargeting.ps1` cannot connect or targets the wrong tenant. **Likely cause:** Graph defaulted to the **Commercial** environment. **Fix:** Connect with `Connect-MgGraph -Environment USGov`. Confirm `graph.microsoft.us` is reachable and allowed on the script host.

J. SSL-inspected endpoint breaks after enabling inspection

Symptom: Everything worked until SSL inspection was turned on near M365 traffic; now Intune/Defender break. **Likely cause:** A no-inspect endpoint (`*.manage.microsoft.us`, `has.spserv.microsoft.com`, `*.dm.microsoft.us`) is being inspected. **Fix:** Ensure `EGRESSG-NoInspect-GCCHigh` is applied **and excluded from inspection**, then re-test. Microsoft does not support inspecting these endpoints.

Escalation checklist (before opening a ticket)

1. Confirm the instance is `USGovGCCHigh` (not `Worldwide`).
2. Confirm portals are `.us` and identity is `login.microsoftonline.us`.
3. Confirm `EGRESSG-NoInspect-GCCHigh` is applied and excluded from inspection.
4. Run `EGRESSG-Validation.ps1` from a real pilot endpoint and capture the PASS/FAIL table.
5. Run `EGRESSG-Discovery.ps1` and confirm "up to date" (or apply the diff).
6. If a capability's Gov behavior is unclear, treat it as **"verify for GCC High"** and check current

Microsoft Gov documentation (see `gcc-high-parity-notes.md`) before escalating.

Best Practices — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

Field-proven practices for running a GCC High allow-list well. They reinforce one idea: **derive, don't hardcode; verify the cloud; keep the blast radius small.**

Query the right cloud — every time

- **Always use the** `USGovGCCHigh` **instance.** The default and `Worldwide` instances are the wrong cloud. Bake the instance name into scripts and scheduled jobs so no one can accidentally query Commercial.
- **Sanity-check the instance in every run.** The scripts confirm the returned `instance` is `USGovGCCHigh` before doing anything else. Keep that check.
- **Trust** `*.us` **/** `*.microsoft.us`**, be suspicious of** `.com`**.** In GCC High, a `.com` M365 endpoint in your derived data almost always means you queried the wrong instance.

Never hardcode Microsoft IP ranges

- Microsoft updates GCC High IP/URL data **monthly**, with additions published **30 days in advance**. A hand-typed list is a scheduled outage.
- Treat `EGRESSG-M365-GCCHigh-Allow` and `EGRESSG-M365-GCCHigh-URLs` as **machine-maintained** objects fed from the Discovery export. Humans review diffs; scripts produce data.
- The only names safe to reference by hand are Microsoft's **documented, stable Gov FQDNs** (portals, identity, Intune/Defender service names) — and even those you cross-check against the derived set.

Respect the SSL-inspection bypass list

- Microsoft **does not support** inspecting `*.manage.microsoft.us`, `has.spserv.microsoft.com`, or `*.dm.microsoft.us`. Inspecting them breaks Intune, Defender, and Endpoint Privilege Management.
- Apply `EGRESSG-NoInspect-GCCHigh` **before** enabling inspection anywhere near M365 traffic, not after users report breakage.
- When Microsoft publishes a new Gov no-inspect endpoint, add it and **verify for GCC High** against current Gov documentation.

Pilot first, always

- Roll out to `EGRESSG-Pilot` before the fleet (Global Requirement G6). Egress changes affect live traffic; a small blast radius is your safety net.
- Keep the pilot representative (a range of OSes, sites, and roles), not just IT machines.
- Validate from a **real endpoint behind the filtered path** — not from the management jump box, which

usually has open egress and hides problems.

Keep the 30-day window

• Schedule Discovery frequently enough (weekly is comfortable) that you always catch additions inside their 30-day advance window.

• Review diffs promptly. The point of the advance window is to apply changes **before** the endpoints go live — not to discover them during an outage.

Name objects consistently and document them

• Use the `EGRESSG-*` object names throughout so the allow-list is auditable and every step references the same objects.

• Keep the Discovery export, the Report output, and the change record together for audit and change control (essential in a government compliance context).

Least privilege and separation

• Discovery, Validation, and Report are **read-only** — run them with reader-level access.

• Reserve device-configuration rights for the person applying the allow-list, and reserve Global Administrator for tenant-wide changes only.

• `Set-ScopeTargeting` and `Remediation` default to `-WhatIf` — keep it that way; require an explicit `-Execute`/`-Execute`-equivalent flag to make changes.

Watch for parity gaps

• Do not assume a Commercial capability exists or behaves identically in GCC High. Where behavior is unconfirmed, treat it as **"verify for GCC High"** and check current Microsoft Gov documentation before relying on it.

• ExpressRoute support, feature availability, and some service FQDNs differ between clouds — derive and

verify rather than carrying over Commercial assumptions.

Have a rollback ready

• Write and approve the rollback (revert allow-list, disable inspection change, restore prior policy) **before** enabling enforcement (Global Requirement G7).

• After any egress-device change, run the regression test in `05-testing.md`. If it fails, roll back first, diagnose second.

Deliverables Summary — Egress Filtering for M365/Intune Endpoints (Government / GCC High)

Everything included in this solution, and exactly what each item does. All artifacts are authored for the **US Government GCC High** cloud and derive Microsoft's required set from the `USGovGCCHigh` instance of the IP/URL web service — **no Microsoft IPs are hardcoded.**

Documents (`source/`)

Document	What it gives you
`01-solution-overview.md`	What the solution does, the governing "query the right instance" rule, and a full **Commercial vs GCC High — what's different** table
`02-requirements.md`	Every prerequisite (R1–R9) with **Verify** and **Implement-if-missing**, GCC High–specific, plus the no-inspect and fixed-FQDN facts
`03-implementation.md`	Step-by-step: derive the Gov set → build the allow-list/URL/no-inspect objects → pilot → validate → expand → keep current, with a gate per phase
`05-testing.md`	Pass/fail gates for each phase and a post-change regression test
`06-troubleshooting.md`	GCC High–specific symptoms, causes, and fixes (wrong instance, `.com` vs `.us`, inspected no-inspect endpoints, 429s)
`07-best-practices.md`	Operating discipline: right cloud, no hardcoding, honor bypass list, pilot first, 30-day window
`08-deliverables-summary.md`	This inventory
`executive-summary.md`	One-page business case for a decision-maker
`gcc-high-parity-notes.md`	Cited reference: instance names, Gov portals/FQDNs, no-inspect endpoints, and Commercial-vs-GCC-High parity gaps
`global-requirements-network.md`	Category-level prerequisites (G1–G7) shared across Network solutions

Scripts (`scripts/`)

All scripts query the `USGovGCCHigh` instance, use `-Environment USGov` for any Graph call, and confirm the returned instance before doing work. Read-only scripts make no changes; change-capable scripts default to `-WhatIf`.

Script	Role	Changes anything?
`EGRESSG-Discovery.ps1`	Derives the current GCC High required set from the web service; writes `EGRESSG-Endpoints-USGovGCCHigh.json` + summary; detects version changes vs `EGRESSG-latestversion.txt` and emits a diff	**No** (read-only)
`EGRESSG-Validation.ps1`	From a real endpoint, tests reachability to representative Gov endpoints per service area; PASS/FAIL	**No** (read-only)
`EGRESSG-Report.ps1`	Produces a dated, point-in-time audit report of the derived set and its categories/ports	**No** (read-only)
`EGRESSG-Remediation.ps1`	Regenerates the corrected derived allow-list export and flags drift; `-WhatIf` **by default**, writes only with `-Execute`	Only with `-Execute`
`EGRESSG-Set-ScopeTargeting.ps1`	Creates/populates the `EGRESSG-Pilot` Entra group via Graph (`-Environment USGov`); `-WhatIf` **by default**, applies only with `-Execute`	Only with `-Execute`

Named objects you will create

Object	Where	Purpose
`EGRESSG-Endpoints-USGovGCCHigh.json`	Script host	Raw derived GCC High set (source of truth for the allow-list)
`EGRESSG-M365-GCCHigh-Allow`	Egress device	IP address group (derived CIDR ranges)
`EGRESSG-M365-GCCHigh-URLs`	Egress device	URL category / bypass for required FQDNs
`EGRESSG-NoInspect-GCCHigh`	Egress device	SSL-inspection bypass list (no-inspect endpoints)
`EGRESSG-Pilot`	Entra	Staged-rollout group
`EGRESSG-latestversion.txt`	Script host	Version + ClientRequestId checkpoint

Compiled deliverables (produced by the build, not by this authoring)

- `EGRESSG-Solution.pdf` — the full solution document.
- `EGRESSG-Scripts.zip` — the five PowerShell scripts packaged for distribution.

Definition of done

A deployment is "done" when: the tenant is confirmed GCC High; the allow-list, URL category, and no-inspect list are built **from the derived** `USGovGCCHigh` **set**; the pilot validated with all checks PASS; the fleet is covered; and the monthly keep-current job runs and reports "up to date" or a reviewed change set.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

EGRESSG-Discovery.ps1

Purpose: Queries the GCC High instance (USGOVGCCHigh) of the Microsoft 365 IP Address and URL web service, splits the result into IPv4/IPv6 ranges, URLs and ports, flags required and no-inspect endpoints, and writes an export plus a human-readable summary. Detects version changes against a checkpoint file and emits an add/remove diff. No Microsoft IPs are hardcoded — everything is derived at run time. Output goes to your-output-folder. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESSG-Discovery.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A JSON export (EGRESSG-Endpoints-USGovGCCHigh.json) and a summary listing *.us sovereign endpoints. Reports 'up to date' when the version is unchanged, otherwise a diff. PASS when the returned instance is USGovGCCHigh and the set is non-empty; FAIL otherwise. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESSG-Discovery.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A JSON export (EGRESSG-Endpoints-USGovGCCHigh.json) and a summary listing *.us sovereign endpoints. Reports 'up to date' when the version is unchanged, otherwise a diff. PASS when the returned instance is USGovGCCHigh and the set is non-empty; FAIL otherwise. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>

EGRESSG-Remediation.ps1

Purpose: Compares a saved allow-list export against the current GCC High set from the USGOVGCCHigh instance and reports drift (ranges/URLs to add or remove). By default it only previews (-WhatIf) and writes nothing. With -Execute it writes a corrected export file for you to import into your egress device — it never edits a firewall or proxy directly. No Microsoft IPs are hardcoded. Point it at your-output-folder and your-previous-export. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESSG-Remediation.ps1 (preview only) powershell -ExecutionPolicy Bypass -File .\EGRESSG-Remediation.ps1 -Execute (write corrected export) (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A drift summary (added/removed ranges and URLs). In preview mode nothing is written. With -Execute a corrected export is written. PASS when the current set is USGovGCCHigh; FAIL on wrong instance. .NOTES White-label — the repeatable way. Read-only by default (-WhatIf); queries the GCC High endpoints web service. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESSG-Remediation.ps1 (preview only) powershell -ExecutionPolicy Bypass -File .\EGRESSG-Remediation.ps1 -Execute (write corrected export) (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A drift summary (added/removed ranges and URLs). In preview mode nothing is written. With -Execute a corrected export is written. PASS when the current set is USGovGCCHigh; FAIL on wrong instance. .NOTES White-label — the repeatable way. Read-only by default (-WhatIf); queries the GCC High endpoints web service. #>

EGRESSG-Report.ps1

Purpose: Derives the current GCC High set from the USGOVGCCHigh instance (or reads an existing Discovery export) and produces a dated audit report grouped by service area and category, with ports, ExpressRoute flags, and the no-inspect endpoints called out. Suitable for change control and government compliance evidence. No Microsoft IPs are hardcoded. Writes to your-output-folder. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESSG-Report.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A dated report file (EGRESSG-Report-<version>.md/.csv) summarizing the required set. PASS when the report is generated from the USGovGCCHigh instance; FAIL on wrong instance. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESSG-Report.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A dated report file (EGRESSG-Report-<version>.md/.csv) summarizing the required set. PASS when the report is generated from the USGovGCCHigh instance; FAIL on wrong instance. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>

EGRESSG-Set-ScopeTargeting.ps1

Purpose: Creates or confirms the EGRESSG-Pilot Microsoft Entra security group and adds specified members, so the new egress allow-list can be piloted before fleet rollout. Connects to the US Government Graph environment (-Environment USGov) — the Commercial default fails on a GCC High tenant. By default it only previews (-WhatIf) and changes nothing; -Execute applies. Provide members as your-user-or-device-object-ids. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESSG-Set-ScopeTargeting.ps1 -GroupName "EGRESSG-Pilot" (preview) powershell -ExecutionPolicy Bypass -File .\EGRESSG-Set-ScopeTargeting.ps1 -GroupName "EGRESSG-Pilot" -Execute (apply) (note the GCC High Graph environment: -Environment USGov; endpoints web-service instance USGOVGCCHigh) EXPECTED RESULTS In preview mode: the planned group create/membership changes are printed, nothing is applied. With -Execute: the group exists and members are added. PASS when connected to the USGov Graph environment; FAIL otherwise. .NOTES White-label — the repeatable way. Read-only by default (-WhatIf); connects to US Gov Graph (-Environment USGov). #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESSG-Set-ScopeTargeting.ps1 -GroupName "EGRESSG-Pilot" (preview) powershell -ExecutionPolicy Bypass -File .\EGRESSG-Set-ScopeTargeting.ps1 -GroupName "EGRESSG-Pilot" -Execute (apply) (note the GCC High Graph environment: -Environment USGov; endpoints web-service instance USGOVGCCHigh) EXPECTED RESULTS In preview mode: the planned group create/membership changes are printed, nothing is applied. With -Execute: the group exists and members are added. PASS when connected to the USGov Graph environment; FAIL otherwise. .NOTES White-label — the repeatable way. Read-only by default (-WhatIf); connects to US Gov Graph (-Environment USGov). #>

EGRESSG-Validation.ps1

Purpose: Derives the current GCC High set from the USGOVGCCHigh instance, then tests TCP/UDP reachability from the machine it runs on to representative Gov endpoints across identity, Intune, Defender/EPM, Exchange, SharePoint, Teams and Graph. Intended to run on a pilot device behind the filtered egress path. No Microsoft IPs are hardcoded; test FQDNs are the documented Gov service names. Reports a PASS/FAIL table. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\EGRESSG-Validation.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh)

EXPECTED RESULTS A table with one row per service area, each PASS (reachable) or FAIL (blocked). Overall PASS only when every required row is PASS. FAIL rows point you at the troubleshooting guide. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>

How to run: powershell -ExecutionPolicy Bypass -File .\EGRESSG-Validation.ps1 (note the GCC High endpoints web-service instance: USGOVGCCHigh) EXPECTED RESULTS A table with one row per service area, each PASS (reachable) or FAIL (blocked). Overall PASS only when every required row is PASS. FAIL rows point you at the troubleshooting guide. .NOTES White-label — the repeatable way. Read-only; queries the GCC High endpoints web service. #>